11.2 Using Prometheus, Jaeger, and Grafana for correlation

Now that you know what correlation is on a conceptual level, let's examine how to implement it. To do so, we will use a concrete example that shows how to do metrics–traces correlation based on exemplars. In the case of metrics–traces correlation, this means embedding a link from the metrics representation to a trace. This is done by inserting a trace ID so that when the metric is rendered in a time series graph, you can click on exemplars and jump directly to a representative trace and navigate its spans. Conceptually, this "embedding of the trace ID in a metric" would look something like the following (using the Prometheus exposition format or, to be more precise, OpenMetrics, since this feature only exists there):

```
a_counter 42.0 # {traceID="a6427ef2021cce03"}
```

Now, we'll move on to our example correlation setup.

11.2.1 Metrics–traces correlation example setup

We will be using Prometheus and Jaeger as backends and Grafana as the frontend. The example service in this example is based on the `echo` service we used in chapter 2. It emits spans that an OpenTelemetry collector routes to Jaeger, and at the same time, the collector scrapes the service `/metrics` endpoint for metrics with embedded trace IDs and makes them available to a Prometheus backend for scraping. You should already know all of these tools and how to configure them, so I will keep it brief here.

All the sources shown are in chapter 11, so change to that directory. First off, the `echo` service, written in Go, emits spans and exposes metrics. In addition, it now also supports OpenMetrics exemplars (http://mng.bz/9DZ0). The source code is in the echo/main.go file; we will only focus on the additions related to how to emit exemplars, and we will only cover the happy path (the error case looks more or less the same, but it returns a `500` code in 40% of the cases, rather than a `200`):

```
invoke, _ := invokes.GetMetricWith(prometheus.Labels{
        "http_status_code": strconv.Itoa(http.StatusOK),
    })
    invoke.(prometheus.ExemplarAdder).AddWithExemplar(
        1,
        prometheus.Labels{"traceID": traceID},
    )
    invokes.WithLabelValues(
        strconv.Itoa(http.StatusOK),
    ).Inc()
    span.SetAttributes(
```

Embeds the trace ID as an exemplar into the invoke counter

Increases the invoke counter with the http_status_code=200 label

Emits the span with the same metadata as used in the metric (HTTP status code)

Besides emitting spans and metrics, with embedded exemplars, there's only one other thing to pay attention to in main.go: the configuration of the Prometheus metrics exposition. We need to make sure we're using the OpenMetrics format, which is done like so: